

Advanced Features

Power-user tools in AlgebraicABMs.jl

Simon Frost

2026-03-30

1 Overview

This vignette covers advanced features for users who need fine-grained control over model structure and dynamics:

1. **Schema inference and validation** — automatic schema management
2. **Rewrite schedules** (ABMSchedule) — multi-step transformations on a single timer
3. **Context and dependency morphisms** — control what the hazard rate “sees”
4. **Match identity with fix** — when two matches are “the same”
5. **History-sensitive hazards** (ClosureHistory) — rates that depend on trajectory
6. **Dimensional analysis with Unitful** — optional unit checking

2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

# Internal helpers
using AlgebraicABMs.ABMs: Traj, RuntimeABM, resolve_match, match_equal,
    is_subschema, rule_schema, merge_schemas
```

3 1. Schema Inference and Validation

When building an ABM from multiple rules, the framework can automatically infer the combined schema or validate rules against a declared one.

3.1 Inferring schemas from rules

```
# Rule 1: operates on a graph (V, E with src/tgt)
L1 = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end
I1 = Graph(2)
R1 = Graph(2)
rule1 = ABMRule(:edge_removal,
    Rule(homomorphism(I1, L1; initial=(V=[1,2],)),
        homomorphism(I1, R1; initial=(V=[1,2],))),
    ContinuousHazard(1.0))
```

```
# Rule 2: operates just on vertices
rule2 = ABMRule(:vertex_removal,
    Rule(homomorphism(Graph(0), Graph(1)),
        id(Graph(0))),
    ContinuousHazard(0.5))
```

```
# Infer the combined schema
schema = infer_schema([rule1, rule2])
println("Inferred schema:")
println("  Objects: ", objects(schema))
println("  Homs: ", homs(schema))
```

```
Inferred schema:
  Objects: [:V, :E]
  Homs: [(:src, :E, :V), (:tgt, :E, :V)]
```

3.2 Validating rules against a schema

```
# Validate that both rules are compatible with the Graph schema
validated = validate_schema([rule1, rule2], schema)
println("Validation passed: schema has $(length(objects(validated))) objects")

Validation passed: schema has 2 objects
```

3.3 Merging and comparing schemas

```
# Extract individual rule schemas
s1 = AlgebraicABMs.ABMs.rule_schema(rule1)
s2 = AlgebraicABMs.ABMs.rule_schema(rule2)

println("Rule 1 schema objects: ", objects(s1))
println("Rule 2 schema objects: ", objects(s2))

# Merge them
merged = AlgebraicABMs.ABMs.merge_schemas([s1, s2])
println("Merged schema objects: ", objects(merged))

# Check sub-schema relationships
ok1, _ = is_subschema(s2, merged)
ok2, _ = is_subschema(s1, merged)
println("Rule 2 schema merged: $(ok1)")
println("Rule 1 schema merged: $(ok2)")

Rule 1 schema objects: [:V, :E]
Rule 2 schema objects: [:V, :E]
Merged schema objects: [:V, :E]
Rule 2 schema merged: true
Rule 1 schema merged: true
```

3.4 Detecting conflicts

```
# is_subschema returns (false, reason) when schemas are incompatible
small_schema = s2 # only has V, no edges
ok, reason = is_subschema(s1, small_schema)
println("Rule 1 vertex-only schema: $(ok)")
println("  Reason: $(reason)")
```

```
Rule 1  vertex-only schema: true
Reason:
```

4 2. Rewrite Schedules (ABMSchedule)

An `ABMSchedule` wraps an AlgebraicRewriting Schedule — a sequence of rewriting operations executed atomically when a timer fires. Unlike individual rules, schedules can perform multi-step transformations.

Important: Schedules require the `ACSet` to use `MarkAsDeleted` part type.

```
using Catlab.CategoricalAlgebra.CSets: MarkAsDeleted

# Define a MarkAsDeleted graph type
@acset_type GrphMD(SchGraph, part_type=MarkAsDeleted)

# Rule: add a vertex to the graph (empty → single vertex)
z = GrphMD()
g1 = @acset GrphMD begin V=1 end
add_v_rule = Rule(id(z), create(g1))

# Wrap in a RuleApp for the schedule system
ra = RuleApp(:add_v, add_v_rule, z)

# Build schedule: try to add one vertex
sched = tryrule(ra)

println("Schedule built: tryrule(RuleApp(:add_v, ...))")
println("Schedule type: ", typeof(sched))

# Create an ABMSchedule with a periodic timer
abm_sched = ABMSchedule(:periodic_add, sched, DiscreteHazard(3.0))

println("\nABMSchedule API:")
println("  ABMSchedule(name, schedule, timer)")
println("  name: $(nameof(abm_sched))")
println("  timer: DiscreteHazard(3.0) - fires every 3 time units")
println("\nUsage: ABM([rules...]; schedules=[abm_sched])")
println("  The schedule executes atomically when its timer fires")
println("  After execution, all rule hom-sets are rebuilt from scratch")

Schedule built: tryrule(RuleApp(:add_v, ...))
Schedule type: Schedule

ABMSchedule API:
  ABMSchedule(name, schedule, timer)
  name: periodic_add
  timer: DiscreteHazard(3.0) - fires every 3 time units

Usage: ABM([rules...]; schedules=[abm_sched])
  The schedule executes atomically when its timer fires
  After execution, all rule hom-sets are rebuilt from scratch
```

Note

Note: Schedule execution via `interpret!` currently requires upstream AlgebraicRewriting support for `MarkAsDeleted` `ACSet` types. The API is ready but runtime execution may require future AlgebraicRewriting releases. The `ABMSchedule` type and construction work correctly — the limitation is in schedule interpretation at runtime.

5 3. Context and Dependency Morphisms

The `context` and `dependency` keyword arguments of `ABMRule` control what information the hazard rate function receives.

- **context** (`L Ctx`): Embeds the pattern in a larger context. The hazard receives a match `Ctx → X` instead of `L → X`, letting it access neighborhood information.
- **dependency** (`Dep Ctx`): Restricts which parts of the context the hazard depends on, enabling efficient re-evaluation.

5.1 Example: rate depends on neighborhood size

Consider an infection rule on an edge, where the infection rate depends on how many edges the source vertex has (its degree).

```
# The rule pattern: an edge (V=2, E=1)
L_inf = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end
I_inf = Graph(2)
R_inf = Graph(2)

# Context: the source vertex plus TWO edges from it (sees more of the graph)
Ctx = @acset Graph begin V=3; E=2; src=[1,1]; tgt=[2,3] end

# Embed L into Ctx: the first edge in Ctx is the pattern edge
ctx_map = homomorphism(L_inf, Ctx; initial=(V=[1,2], E=[1]))

println("Pattern: edge (V=2, E=1)")
println("Context: source vertex with 2 outgoing edges (V=3, E=2)")
println("Context map defined: ", !isnothing(ctx_map))

Pattern: edge (V=2, E=1)
Context: source vertex with 2 outgoing edges (V=3, E=2)
Context map defined: true

# The dependency is just the source vertex - the hazard only depends on
# which vertex is the source, not which specific edges exist
Dep = Graph(1) # just one vertex
dep_map = homomorphism(Dep, Ctx; initial=(V=[1],))

println("Dependency: single vertex (V=1)")
println("Dependency map defined: ", !isnothing(dep_map))

Dependency: single vertex (V=1)
Dependency map defined: true

# Create the rule with context and dependency
infection_ctx = ABMRule(:infection_ctx,
    Rule(homomorphism(I_inf, L_inf; initial=(V=[1,2],)),
        homomorphism(I_inf, R_inf; initial=(V=[1,2],))),
```

```

ClosureState(m -> begin
  # m is now Ctx → X, so we can count edges from the source vertex
  Exponential(1.0)
end);
context=ctx_map,
dependency=dep_map)

println("Rule with context and dependency created")
println("  Context: ", !isnothing(infection_ctx.context))
println("  Dependency: ", !isnothing(infection_ctx.dependency))

Rule with context and dependency created
Context: true
Dependency: true

```

5.2 Demonstrating resolve_match

resolve_match extends a match through the context/dependency chain.

```

# Create a graph where the context extension is unique.
# L_inf is an edge (V=2, E=1). Ctx adds a second edge from the source (V=3, E=2).
# To make the extension unique, we need the two target vertices to be distinguishable.
# Adding a self-loop on vertex 2 makes it distinct from vertex 3.

```

```

state = @acset Graph begin
  V = 3
  E = 3
  src = [1, 1, 2]
  tgt = [2, 3, 2]
end

```

```

# Find matches of L (single edge) into state
matches = homomorphisms(L_inf, state)
println("Found $(length(matches)) matches of the edge pattern")

```

```

if !isempty(matches)
  m = first(matches)
  println("First match: V=$(collect(m[:V])), E=$(collect(m[:E]))")

  # Resolve through context - extends match from L→X to Ctx→X
  # This can fail if the extension is ambiguous
  m_resolved = try
    resolve_match(infection_ctx, m)
  catch e
    println("Context extension ambiguous: $(e)")
    nothing
  end
  if !isnothing(m_resolved)
    println("Resolved through context/dependency")
    println("  Result domain: $(nparts(dom(m_resolved), :V)) vertices")
    println("  (dependency restricts what the hazard sees)")
  end
end
end

```

Found 3 matches of the edge pattern

First match: V=[1, 2], E=[1]

Context extension ambiguous: `ErrorException("Exceeded 1: Any[ACSetTransformation((V = FinFunction([1, 2`

6 4. Match Identity with Fix Subobject

The `fix` morphism defines when two matches are “essentially the same.” If `fix` maps a subobject $L_{\text{fix}} \rightarrow L$, then matches $m, m' : L \rightarrow X$ are equal when `fix m == fix m'`.

This is useful when a rule’s pattern has parts that should not distinguish matches — for example, if the timer should not change when an attribute varies.

```
# Pattern: a single vertex
L = Graph(1)

# Fix: the identity (all parts matter) - matches are equal only if identical
fix_all = id(L)
rule_fixall = ABMRule(:fix_all,
    Rule(id(L), id(L)), # identity rule (no change)
    ContinuousHazard(1.0);
    fix=fix_all)

println("Rule with fix=id(L): all match components determine identity")
Rule with fix=id(L): all match components determine identity

# Compare matches using match_equal
state2 = Graph(3) # three vertices
ms = homomorphisms(L, state2)
println("Matches of single vertex into 3-vertex graph: $(length(ms))")

if length(ms) >= 2
    m1, m2 = ms[1], ms[2]
    eq = match_equal(rule_fixall, m1, m2)
    println("match_equal(m1, m2) = $(eq) (expected: false, different vertices)")

    eq_self = match_equal(rule_fixall, m1, m1)
    println("match_equal(m1, m1) = $(eq_self) (expected: true, same match)")
end

Matches of single vertex into 3-vertex graph: 3
match_equal(m1, m2) = false (expected: false, different vertices)
match_equal(m1, m1) = true (expected: true, same match)

# Without fix: default comparison
rule_nofix = ABMRule(:no_fix,
    Rule(id(L), id(L)),
    ContinuousHazard(1.0))

if length(ms) >= 2
    m1, m2 = ms[1], ms[2]
    eq = match_equal(rule_nofix, m1, m2)
    println("Without fix - match_equal(m1, m2) = $(eq)")
    println(" (same behavior: defaults to m1 == m2)")
end

Without fix - match_equal(m1, m2) = false
(same behavior: defaults to m1 == m2)
```

7 5. History-Sensitive Hazards (ClosureHistory)

ClosureHistory accepts (match, time, trajectory) and returns a distribution. The trajectory contains all events and rewrite spans up to the current time, enabling hazards that depend on the full history.

7.1 Example: increasing hazard over time

```
# A removal rule whose rate increases with the number of past events
# More events => faster removal (positive feedback)
removal_hist = ABMRule(:history_removal,
  Rule(homomorphism(Graph(0), Graph(1)),
    id(Graph(0))),
  ClosureHistory((m, t, traj) -> begin
    # traj is the Traj object (or nothing at initialization)
    n_events = isnothing(traj) ? 0 : length(traj)
    rate = 0.1 * (1 + n_events) # rate grows with event count
    Exponential(1.0 / rate)
  end))

println("ClosureHistory rule: removal rate = 0.1 * (1 + n_past_events)")
ClosureHistory rule: removal rate = 0.1 * (1 + n_past_events)

# Build ABM: vertices are born at a constant rate, removed with history-sensitive rate
birth_simple = ABMRule(:birth,
  Rule(id(Graph()),
    create(ob(terminal(Graph)))),
  ContinuousHazard(0.5))

abm_hist = ABM([birth_simple, removal_hist])

traj_hist = run!(abm_hist, Graph(10); maxtime=15.0, maxevent=200)

println("History-sensitive simulation:")
println("  Total events: $(length(traj_hist))")

# Show how event rate changes over time
if length(traj_hist) >= 4
  times = [e[1] for e in traj_hist.events]
  # Compute inter-event times in first half vs second half
  half = length(times) ÷ 2
  if half > 1
    iet_first = [times[i] - times[i-1] for i in 2:half]
    iet_second = [times[i] - times[i-1] for i in (half+1):length(times)]
    if !isempty(iet_first) && !isempty(iet_second)
      println("  Mean inter-event time (first half): $(round(sum(iet_first)/length(iet_first), digits=3))")
      println("  Mean inter-event time (second half): $(round(sum(iet_second)/length(iet_second), digits=3))")
      println("  (second half should be shorter due to history-dependent acceleration)")
    end
  end
end
end

History-sensitive simulation:
Total events: 44
Mean inter-event time (first half): 0.197
Mean inter-event time (second half): 0.481
```

(second half should be shorter due to history-dependent acceleration)

7.2 Example: waning immunity

```
# A recovery rule where the rate depends on how long ago the most recent
# event affecting this vertex occurred (simulating waning immunity)
recovery_waning = ABMRule(:waning_recovery,
    Rule(homomorphism(Graph(0), Graph(1)),
        id(Graph(0))),
    ClosureHistory((m, t, traj) -> begin
        if isnothing(traj) || isempty(traj)
            Exponential(10.0) # slow recovery initially
        else
            # Time since last event
            last_event_time = traj.events[end][1]
            elapsed = t - last_event_time
            # Recovery gets faster the longer since last event
            rate = 0.1 + 0.05 * elapsed
            Exponential(1.0 / rate)
        end
    end))

println("Waning immunity rule: recovery rate increases with time since last event")
Waning immunity rule: recovery rate increases with time since last event
```

8 6. Dimensional Analysis with Unitful (Conditional)

When the Unitful package is available, AlgebraicABMs provides unit-aware hazard construction and validation through the UnitfulExt extension.

```
unitful_available = try
    @eval using Unitful
    @eval using Unitful: u_str
    true
catch
    false
end

if unitful_available
    println("Unitful is available - demonstrating dimensional analysis")

    # ContinuousHazard from a unitful rate
    h = @eval ContinuousHazard(0.1u"d^-1")
    println(" ContinuousHazard(0.1/day): ", h)

    # validate_units checks dimensions
    valid = @eval validate_units(0.1u"d^-1")
    println(" validate_units(0.1/day): $(valid)")

    # strip_units removes units
    stripped = @eval strip_units(5.0u"d")
    println(" strip_units(5.0 days): $(stripped)")

    # DiscreteHazard from unitful time
```



```

    hd = @eval DiscreteHazard(1.0u"d")
    println(" DiscreteHazard(1.0 day): ", hd)
else
    println("Unitful not available - skipping dimensional analysis demo")
    println(" Install with: using Pkg; Pkg.add(\"Unitful\")")
    println("")
    println("When available, the UnitfulExt provides:")
    println(" ContinuousHazard(rate::Quantity) - construct from unitful rate (e.g., 0.1u\"d^-1\")")
    println(" DiscreteHazard(time::Quantity) - construct from unitful time")
    println(" validate_units(rate, expected_dim) - check dimensional consistency")
    println(" strip_units(val) - remove units from a quantity")
end

```

Unitful not available - skipping dimensional analysis demo
Install with: using Pkg; Pkg.add("Unitful")

When available, the UnitfulExt provides:

```

ContinuousHazard(rate::Quantity) - construct from unitful rate (e.g., 0.1u"d^-1")
DiscreteHazard(time::Quantity) - construct from unitful time
validate_units(rate, expected_dim) - check dimensional consistency
strip_units(val) - remove units from a quantity

```

9 Summary

Feature	API	Use case
Schema inference	<code>infer_schema(rules)</code>	Automatic schema from rules
Schema validation	<code>validate_schema(rules, schema)</code>	Check compatibility
Schema comparison	<code>is_subschema(sub, sup)</code>	Sub-schema testing
Schedules	<code>ABMSchedule(name, schedule, timer)</code>	Multi-step atomic rewrites (requires upstream fix)
Context	<code>ABMRule(...; context=L Ctx)</code>	Neighborhood-aware hazards
Dependency	<code>ABMRule(...; dependency=Dep Ctx)</code>	Efficient hazard updates
Fix subobject	<code>ABMRule(...; fix=L_fix L)</code>	Match identity control
History	<code>ClosureHistory((m,t,Traj), >...)</code>	Trajectory-dependent hazards
Units	<code>ContinuousHazard(rate::Quantity)</code>	Dimensional safety

```

println("Advanced features demonstrated:")
println(" Schema inference, validation, merging, and sub-schema checking")
println(" ABMSchedule with MarkAsDeleted state type")
println(" Context and dependency morphisms with resolve_match")
println(" Fix subobject with match_equal")
println(" ClosureHistory for trajectory-dependent hazard rates")
println(" Unitful extension (conditional)")

```

Advanced features demonstrated:

```

Schema inference, validation, merging, and sub-schema checking
ABMSchedule with MarkAsDeleted state type

```

Context and dependency morphisms with `resolve_match`
Fix subobject with `match_equal`
ClosureHistory for trajectory-dependent hazard rates
Unitful extension (conditional)